Lab Report: Dataset Creation and Labeling (Cat Object Detection)

1. Introduction

In this lab, the objective was to build a small portion of a real-world object detection dataset. The dataset focuses on detecting the **Cat** class using images from the **Open Images V6** dataset. Each student in the class was responsible for downloading, labeling, and verifying a unique subset of images.

This approach simulates how machine learning teams collaboratively build large datasets. The workflow includes collecting metadata, selecting a subset of images, downloading them, generating bounding box labels, and performing quality verification before submission.

---

2. Dataset Source

The dataset used in this lab was **Open Images V6**, a large public dataset containing millions of images and annotations for object detection.

Three metadata files were required:

- **class-descriptions-boxable.csv** – maps machine-readable label IDs to class names.
- **oidv6-train-annotations-bbox.csv** – contains bounding box annotations for training images.
- **train-images-boxable-with-rotation.csv** – contains image metadata and download URLs.

These files were stored in the folder:

```
openimages_v6/
```

The bounding box annotation file is large (~2.2GB) and contains the coordinates for objects detected in images.

---

3. Building the Master Image List

The first step was to generate a master list of images containing at least one valid **Cat** bounding box.

The following command was used:

```
python build_master_list.py --class-descriptions openimages_v6/class-descriptions-boxable.csv --bboxes openimages_v6/oidv6-train-annotations-bbox.csv --out master_cat_imageids.txt
```

This script filtered the annotations dataset to keep only:

- Cat class annotations
- Non-group objects
- Non-depiction objects

After filtering, the script extracted unique ImageIDs and sorted them lexicographically to produce a consistent master list.

---

4. Dataset Partitioning

To ensure that each student worked on a **unique and non-overlapping subset**, the dataset was partitioned deterministically.

Each student calculated their **rank** within the class roster and used the following rule:

```
ImageID[rank], ImageID[rank + N], ImageID[rank + 2N], ...
```

Where:

- **rank** = student's position in the roster
- **N** = number of students

The command used was:

```
python pick_my_imageids.py --master master_cat_imageids.txt --rank 33 --N 6 --k 100 --out my_imageids.txt
```

This generated a file containing **100 ImageIDs** assigned to the student.

---

5. Image Downloading

Images corresponding to the selected ImageIDs were downloaded using the metadata file containing image URLs.

Command used:

```
python download_images.py --image-metadata openimages_v6/train-images-boxable-with-rotation.csv --imageids my_imageids.txt --out images
```

The script automatically downloaded images and stored them in the **images/** folder.

If an image was already present locally, it was skipped to avoid redundant downloads.

---

6. Generating YOLO Labels

The bounding box annotations from Open Images were converted to **YOLO format**.

Open Images format uses corner coordinates:

```
XMin, XMax, YMin, YMax
```

These were converted to YOLO format:

```
x_center = (XMin + XMax) / 2
y_center = (YMin + YMax) / 2
width    = XMax - XMin
height   = YMax - YMin
```

Each label file contained entries in the format:

```
0 x_center y_center width height
```

where **0 represents the Cat class**.

Command used:

```
python export_yolo_labels.py --class-descriptions openimages_v6/class-descriptions-boxable.csv --bboxes openimages_v6/oidv6-train-annotations-bbox.csv --imageids my_imageids.txt --out labels
```

---

7. Dataset Verification

Before submission, the dataset was verified using an automated script.

Command used:

```
python verify_yolo_dataset.py --images images --labels labels --imageids my_imageids.txt
```

The script checked that:

- Every ImageID had a corresponding image
- Every image had a label file
- Label format was correct
- Bounding box coordinates were valid

The verification process ensured that the dataset was ready for object detection model training.

---

8. Challenges and Observations

One challenge encountered during the lab was that some images failed to download due to unavailable URLs or network issues. This is common when working with large public datasets.

However, the verification script helped identify missing images and ensured that the dataset maintained structural consistency.

Another observation was the importance of automated validation when working with labeled datasets. Even small formatting errors in label files can cause training failures in machine learning models.

10. Bias Analysis and Tricky Labeling Cases

When building object detection datasets, it is important to consider potential **dataset bias** and difficult labeling scenarios. Bias can occur when the dataset does not represent real-world diversity or when certain types of images appear more frequently than others.

One possible bias in the Cat dataset is **environment bias**. Many images in large datasets contain cats in indoor environments such as homes or furniture. As a result, a model trained on this dataset might perform better at detecting cats indoors than in outdoor environments.

Another type of bias is **appearance bias**. Some breeds of cats may appear more frequently in the dataset than others. For example, common domestic cats may be overrepresented, while rare breeds or unusual appearances may appear less often. This imbalance could affect the model's ability to generalize to all cat types.

Several **tricky labeling cases** were also observed during dataset creation:

1. Occlusion
In some images, the cat is partially hidden behind objects such as furniture, plants, or other animals. This makes it difficult to determine the exact bounding box.

Example:
A cat sitting behind a chair where only the head and part of the body are visible.

2. Multiple Cats in One Image
Some images contain several cats at different positions or distances. Each cat must be labeled with a separate bounding box, which increases labeling complexity.

Example:
A group of cats sitting together in a room or outside in a park.

3. Small or Distant Objects
Cats that appear very small in the image can be difficult to detect and label accurately. Small bounding boxes may lead to less precise detection during training.

Example:
A cat far away in the background of a large outdoor scene.

4. Unusual Poses
Cats sometimes appear in unusual positions such as jumping, stretching, or partially out of the frame. This makes defining the correct bounding box more challenging.

Example:
A cat jumping in mid-air or partially outside the image boundary.